si va bene, ma : ci sono altri argomenti che potrebbero essere dichiarati in docker_run_extra_args ed effettivamente passati al comando docker run ? se si, anche per questi sarebbe necessario prevedere la gestione in project.yaml ?

Sì: `docker-run-extra-args` può contenere molti altri argomenti oltre alle porte. E sì: quelli che rappresentano esigenze del progetto e che vuoi applicare indifferentemente in legacy e Compose devono avere una dichiarazione comune in `project.yaml`.

La distinzione importante è fra **configurazione del progetto** e opzioni operative dell'orchestratore: non conviene trasformare `project.yaml` in una copia completa della CLI di Docker.

1. Che cosa consente il meccanismo attuale

Il tuo resolver accetta, da ambiente o file, una stringa di opzioni per `docker run`. `run.sh` la suddivide in argomenti e la inserisce prima dell'immagine.

Può quindi veicolare, per esempio:

```
-e UV_SYNC_ALWAYS=1
--env-file ./development.env
-p 8080:8000
-v /host/data:/data
--device /dev/ttyUSB0
--add-host host.docker.internal:host-gateway
--cap-add SYS_PTRACE
--security-opt seccomp=unconfined
--shm-size 1g
--memory 2g
--cpus 2
```

Docker offre opzioni per ambiente, mount, rete, risorse, dispositivi, sicurezza e override dei default dell'immagine.[1]

Questo non significa che il parser attuale gestisca qualsiasi valore correttamente: lo splitting per whitespace non conserva in generale il quoting di valori contenenti spazi. Nel nuovo percorso è preferibile generare direttamente un array di argomenti, non una stringa di comando.

2. Che cosa dovrebbe andare in `project.yaml`

La regola proposta è:

> Se un'impostazione descrive ciò di cui il progetto ha bisogno a runtime, deve essere dichiarabile in forma strutturata e tradotta da entrambi i backend.

Ecco una prima classificazione.

Esigenza	Opzioni legacy tipiche	Dichiarazione comune proposta
Variabili d'ambiente	`--env`, `-e`	`runtime.environment`
File di variabili	`--env-file`	`runtime.env_files`
Porte pubblicate	`--publish`, `-p`	`runtime.ports`
Mount aggiuntivi	`--mount`, `--volume`, `-v`	`runtime.mounts`
Filesystem temporanei	`--tmpfs`	`runtime.tmpfs`
Dispositivi host	`--device`	`runtime.devices`
Host aggiuntivi	`--add-host`	`runtime.extra_hosts`
DNS personalizzati	`--dns`, `--dns-search`	`runtime.dns`, `runtime.dns_search`
Memoria condivisa	`--shm-size`	`runtime.shm_size`
Limiti di risorse	`--memory`, `--cpus`, `--pids-limit`	`runtime.resources`
Capability	`--cap-add`, `--cap-drop`	`runtime.security.cap_add`, `cap_drop`
Opzioni di sicurezza	`--security-opt`, `--read-only`	`runtime.security`
Gruppi supplementari	`--group-add`	`runtime.group_add`
GPU	`--gpus`	`runtime.gpus`, con controllo del supporto

Docker e Compose offrono configurazioni per molte di queste esigenze; la traduzione deve comunque essere definita e testata campo per campo, senza presumere equivalenza universale fra tutte le opzioni.[2] [1]

I nomi sotto `runtime` sono una proposta di schema di `docker-dev`, non nomi già implementati né necessariamente identici alle chiavi Compose.

3. Esempio di configurazione comune

Un progetto potrebbe dichiarare:

```
schema_version: 1

runtime:
  environment:
    UV_SYNC_ALWAYS: "1"
    APP_MODE: development
```

```
  env_files:
    - ./development.env

  ports:
    - target: 8000
      published: "8080"
      host_ip: "127.0.0.1"
      protocol: tcp

  mounts:
    - type: bind
      source: ./data
      target: /data
      read_only: false

  extra_hosts:
    host.docker.internal: host-gateway

  shm_size: 1g

  resources:
    memory: 2g
    cpus: 2
```

Il progetto dichiara l'intenzione una sola volta:

```
project.yaml
     ↓
risoluzione e validazione comune
     ├─ legacy  → array di opzioni docker run
     └─ Compose → configurazione del servizio dev
```

Nel backend legacy si produrrebbero argomenti come:

```
runtime_args=(
  --env "UV_SYNC_ALWAYS=1"
  --env "APP_MODE=development"
  --env-file "$PROJECT_DIR/development.env"
  --publish "127.0.0.1:8080:8000/tcp"
  --mount "type=bind,source=$PROJECT_DIR/data,target=/data"
  --add-host "host.docker.internal:host-gateway"
  --shm-size "1g"
  --memory "2g"
  --cpus "2"
)
```

Nel backend Compose si produrrebbero le corrispondenti proprietà di `services.dev`.

Occorre definire anche la semantica comune: per esempio, i percorsi relativi vanno risolti rispetto alla directory del progetto e la precedenza tra `environment` ed `env_files` deve essere identica nei due backend.

4. Che cosa non dovrebbe diventare un'opzione libera del progetto

Alcuni argomenti riguardano il controllo del ciclo di vita o identità che `docker-dev` già gestisce.

Opzione	Gestione proposta
`--name`	Derivata dall'orchestratore
Riferimento immagine	Risolto dalla configurazione del linguaggio e dagli override previsti
`--rm`	Politica della modalità effimera
`-i`, `-t`	Modalità interattiva oppure esecuzione batch
`-d`	Operazione di avvio, non requisito applicativo
`--restart`	Politica di un servizio residente, non default della sessione effimera
Comando da eseguire	Argomenti dopo `--`, preservati come argv

Questi aspetti possono essere configurabili, ma attraverso opzioni dedicate di `docker-dev`, non mediante un passthrough indiscriminato.

Anche `--user`, `--workdir` e `--entrypoint` richiedono una gestione speciale: Docker consente di sovrascrivere i default dell'immagine, ma nel tuo ambiente Python questi valori partecipano all'inizializzazione del virtualenv e della shell.[1]

Per la prima versione li terrei come invarianti del template di linguaggio. Un eventuale override dovrebbe essere esplicito, validato e documentato come potenzialmente incompatibile con il normale setup.

5. Non è necessario supportare tutto subito

Introdurre `project.yaml` non impone di implementare ogni flag Docker.

È meglio definire un insieme supportato, versionato:

Prima versione

- ambiente e file ambiente;
- porte;
- mount aggiuntivi;
- host aggiuntivi;
- memoria condivisa e limiti di risorse.

Estensioni successive, oppure anticipate se necessarie

- dispositivi e gruppi supplementari;
- capability e opzioni di sicurezza;
- GPU;

- DNS e reti personalizzate;
- tmpfs e ulteriori impostazioni.

Un campo non supportato deve produrre un errore chiaro. Non deve essere ignorato in uno dei due backend.

Per configurazioni come rete host, modalità privilegiata o override dell'entrypoint non basta una traduzione sintattica: occorre stabilire se siano compatibili con le invarianti dello stack.

6. Come migrare `docker-run-extra-args`

Proporrei questo obiettivo:

```
project.yaml = fonte canonica delle esigenze runtime del progetto
```

Durante la transizione:

1. Il percorso legacy continua ad accettare gli extra args esistenti.
2. Le funzionalità migrate vengono dichiarate in `project.yaml`.
3. Se la stessa impostazione compare in entrambe le fonti, `docker-dev` segnala il conflitto.
4. In Compose, gli extra args non traducibili non vengono ignorati: l'avvio si interrompe con indicazione della configurazione da migrare.
5. Il passthrough legacy viene eventualmente deprecato, senza rimuoverlo subito.

`compose.user.yaml` può restare disponibile per personalizzazioni specifiche Compose, ma non dovrebbe essere la fonte ordinaria delle impostazioni che devono funzionare anche in legacy.

7. Conseguenza sulla CR

La modifica non riguarda più soltanto le porte. Occorre introdurre un componente comune di risoluzione del runtime:

```
configurazione di linguaggio
          +
configurazione project.yaml
          +
override CLI espliciti
          ↓
runtime normalizzato
          ├─ backend docker run
          └─ backend Compose
```

Il resolver dei servizi rimane distinto: seleziona PostgreSQL, Redis ecc. e ne risolve configurazione e integrazione. Non deve diventare responsabile di tutte le opzioni del container di sviluppo.

Quindi la risposta è: sì, le altre esigenze runtime portabili devono essere gestite in `project.yaml`; non tutti gli argomenti Docker, però, devono essere esposti al progetto. La CR

dovrebbe definire uno schema comune limitato e verificabile, lasciando all'orchestratore identità, ciclo di vita e modalità di esecuzione.

⁂

1. https://docs.docker.com/engine/containers/run/
2. https://docs.docker.com/reference/compose-file/services.md
3. https://github.com/meibraransari/Docker-Zero-to-Hero/tree/main/Day-03_Docker_run_command_with_all_possible_options
4. https://safeguard.sh/resources/blog/docker-privileged-mode-risks
5. https://docs.docker.com/reference/compose-file/services/
6. https://docs.docker.com/reference/cli/docker/container/run/
7. https://docs.docker.com/reference/compose-file/
8. https://docs.docker.com/reference/compose-file/build/
9. https://github.com/compose-spec/compose-spec/blob/master/spec.md
10. https://docs.nvidia.com/ai-workbench/user-guide/latest/reference/projects/compose-reference.html
11. https://www.okteto.com/docs/1.42/reference/docker-compose/
12. https://stackharbor.com/en/knowledge-base/docker-run-essentials/
13. https://techearl.com/docker-run-cheat-sheet
14. https://freereference.com/sheet/docker-run-options
15. https://oneuptime.com/blog/post/2026-02-08-how-to-use-docker-compose-devices-configuration/view
16. https://stackoverflow.com/questions/37904682/how-do-i-use-docker-environment-variable-in-entrypoint-array